01 / PRODUCT & PURPOSE

Agree on the scope. Before the work.

ScopeMatch helps freelancers and small-business clients reveal different expectations and confirm one shared scope before implementation starts.

THE SAME BRIEF: “WE NEED ONLINE BOOKING.”

FREELANCER IMAGINES

A conversation on WhatsApp.

≠

CLIENT IMAGINES

An instant reservation.

AFTER DISCUSSION AND BOTH CONFIRMATIONS

A booking request that staff approve.

AGREED

Illustration of the implemented booking-question flow. ScopeMatch records the decision; it does not operate the booking service.

Who it serves

A freelancer building a small-business website and the business owner hiring them. The freelancer uses a desktop; the client can participate on a phone through a private link, without an account.

Why it helps

Approving “booking” does not establish what booking means. Independent answers expose the difference before a proposal becomes implementation work.

The core idea

Both people answer the same questions privately. Once both submit, ScopeMatch reveals their answers, supports negotiation and records the exact scope version they both confirm.

EIGHT CURATED TOPICS + OPTIONAL AI QUESTIONS

BOOKING · LANGUAGES · EDITABLE CONTENT · LAUNCH CONTENT · PAYMENTS · DELIVERY · CONTACT · DOMAIN & HOSTING

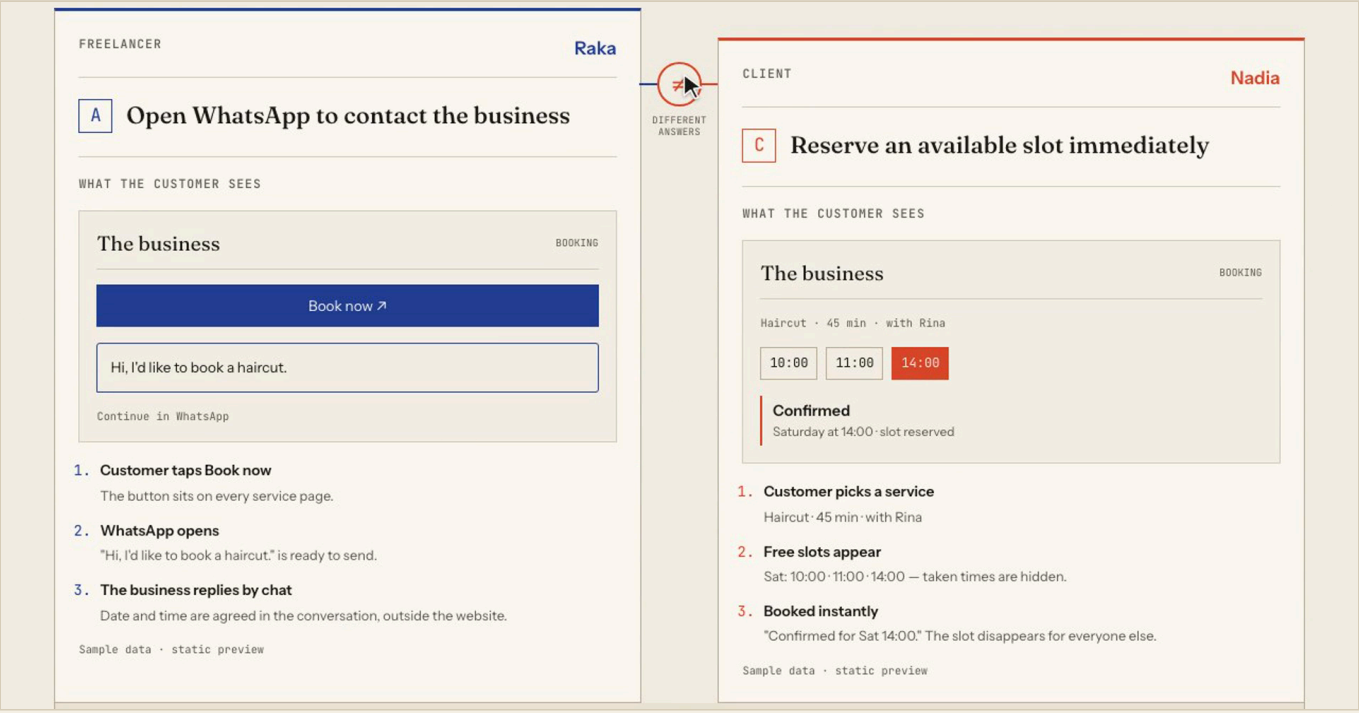
PUBLIC SOURCE / MIT LICENSE

github.com/fadillaoegi/scopematch-hacketon

02 / APPLICATION FLOW

From hidden answers to a visible difference.

The working demo follows a fictional surf-school project with Raka as freelancer and Nadia as client.



Actual app footage: different booking choices, with static customer-view previews. The sample previews do not make reservations.

01 Prepare

Enter the brief and both names. Select relevant curated questions; optionally review AI drafts.

02 Lock and invite

Lock the questions, keep the freelancer link and share the client's private link.

03 Answer separately

Each person answers and seals their choices. The server hides the other person's answers.

04 Reveal and compare

After both submit, see matches, differences and answers needing clarification.

05 Reach decisions

Propose, counter-propose and confirm. Matching answers also require both confirmations.

06 Confirm one version

Review the assembled scope. Both people confirm the same saved version; it becomes locked.

One agreed record, two export formats

The scope contains decisions, included work, exclusions, client responsibilities, acceptance criteria and open questions. Download Markdown or use **Print / Save as PDF**. The demo clicks the print action and opens the actual two-page PDF.

Revising the scope before agreement requires fresh confirmations. Names and times are recorded, not identity-verified. The proof of concept does not provide verified signatures, payments, chat or team management.

03 / AI, ARCHITECTURE & EVIDENCE

AI supports the draft. People own the decision.

Optional Google Gemini suggestions extend the curated library during Setup, before the questions are locked.

01 / REQUEST

Explicit action

The freelancer requests suggestions. The screen discloses that the brief is sent to Gemini.

02 / REVIEW

Up to three drafts

Inspect each option's scope details, then add or discard. The backend validates accepted cards.

03 / DECIDE

Both people confirm

AI never answers, negotiates or approves. Accepted questions follow the same human agreement flow.

Core brief matching is deterministic and needs no API key. Optional AI requires a server-side key and network access. Drafts can be unsuitable; human review is essential. The app also discloses the provider's potential free-tier use of submitted content.

Clean Architecture

Backend / Go + SQLite

Adapters: HTTP, SQLite and Gemini.

Application: `internal/scope` owns use cases and repository / suggestion ports.

Domain: persistence-neutral entities; pure scenario matching in the library.

Frontend / React + TypeScript

Presentation: responsive components and browser interactions.

Application: gateway contract and semantic errors.

Domain: framework-independent models. HTTP is an injected adapter.

Dependencies point inward. Backend rule checks and writes share one repository transaction. The server enforces answer privacy, role access, current-version confirmation and the agreed-state lock. Architecture details.

Verification

44

checks passed in the recorded desktop / phone browser run.

33

TypeScript files passed frontend architecture checks.

128s

application demo, including live AI and actual Markdown / PDF output.

Passed on 17 October: Go tests, `go vet ./...` and frontend production build. Existing browser/media evidence covers sealing, counter-proposals, revised versions, both confirmations and agreed exports.

Automated demonstrations use fictional participants. A real-user test protocol exists, but no completed real-user results or measured commercial impact is claimed. Run instructions · Verification record.

04 / REQUIREMENTS & LINKS

A clear project. A complete entry.

Submission requirements checked against the official competition pages on 17 October 2026. [1, 2]

REQUIREMENT	EVIDENCE / REMAINING ACTION	STATUS
New work + Learn Skill Pack [2]	Planning and build history are recorded; the repository was created within the competition period.	EVIDENCED
Working software [1]	Brief, sealed answers, comparison, negotiation, versioned agreement and exports work end to end.	WORKING
Public source + plans [2]	scope.md, prd.md and spec.md are in the public MIT-licensed repository. README documents installation and testing.	AVAILABLE
English description [1, 2]	App, this brief and narration text are English. Review the actual form and add personal learning answers.	REVIEW FORM
Video under 3 minutes; public YouTube / Vimeo URL [1]	The local app-only master is 2:08. Separate intro + closing would make 2:22. Final voiced video and public URL are unverified.	UPLOAD
Testing access [1]	Public source and local run instructions exist. No permanent hosted deployment is recorded. Curated use needs no AI key.	VERIFY LINKS
Participant declarations + entry [1]	Registration was reported complete. Eligibility, ownership, required form fields and the final submission receipt need entrant verification.	ENTRANT

Deadline: 27 October 2026, 04:00 WIB
Equivalent to 26 October 2026, 17:00 EDT. Upload the final video, verify anonymous access, complete the actual Devpost form and retain its confirmation. [2]

Participant and media checks

The overview states 18+; full rules apply local majority age, residency / sanctions and conflict exclusions. Confirm team authority, ownership and third-party permissions personally. Read the full terms. [1, 2]

Judging and document scope

Design, potential impact, originality and presentation carry equal weight. This PDF is supplemental; published rules list no required PDF or maximum PDF page count. [1]

SOURCES & PROJECT REFERENCES

[1] Official rules: learn-ai-basics.devpost.com/rules

[2] Event overview: learn-ai-basics.devpost.com

Project: github.com/fadillaoegi/scopematch-hacketon
Run / test guide · Architecture · Delivery record · Third-party notices

Built with the Devpost Learn Skill Pack, Claude Code and Codex. Snapshot: 4b7b606. Final upload and entry remain unverified.